

Operating system principles 3

Paul Blondel

CNAM

September 1st, 2018

'(Kenneth) Thompson and (Dennis) Ritchie were among the first to realize that hardware and compiler technology had become good enough that an entire operating system could be written in C, and by 1978 the whole environment had been successfully ported to several machines of different types.'

Eric S. Raymond

- 1 Introduction
- 2 Contiguous allocation of the physical memory
- 3 Non-contiguous memory allocation
- 4 Virtual memory

- 1 Introduction
- 2 Contiguous allocation of the physical memory
- 3 Non-contiguous memory allocation
- 4 Virtual memory

Recall

The *main memory* is where *running software* and *opened documents* are. It's **important to well manage the main memory**:

- The **more software we launch**, the **more data** we use
- If **not enough memory**: cannot execute new softwares

In the *main memory* for each process:

- there is a *starting* and an *ending address*
- there is an associated *allocated data*
- the allocated data is *protected from other processes*

Virtual memory

The *virtual memory* is stored in the *mass storage memory* and is used when the *main memory* is filled

Physical memory

The *physical memory* is stored in the *main memory*, you cannot have more physical memory than the main memory. ¹

Logic addresses

These are the **addresses within the executable** when it is in the *mass storage memory* (before getting loaded in the main memory)

Physical addresses

These are the **addresses within the executable** when it is loaded in *main memory*

¹The physical memory space is where software and opened documents are.

When a user launches an *executable*:

- The **loader loads the executable in main memory**
 - ▶ The loader **must find a place big enough** in main memory
 - ▶ Memory is allocated at this location
 - ▶ *logical addresses* are **converted** into *physical addresses*

When a user launches an *executable*:

- The **loader** loads the executable in main memory
 - ▶ The loader **must find a place big enough** in main memory
 - ▶ Memory is allocated at this location
 - ▶ *logical addresses* are **converted** into *physical addresses*

When the *process* has terminated its execution:

- The *allocated memory* is **freed from the memory**
- This memory is **available again** for other *processes*

Two approaches

There are *two approaches* to **allocate** and **release** memory:

- ① **Contiguous approach**: one program = one **unique** and **inseparable** memory **partition**
- ② **Non-contiguous approach**: one program = set of **separable parts** dispatched in memory

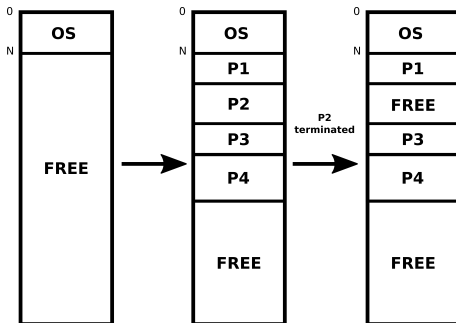
- 1 Introduction
- 2 Contiguous allocation of the physical memory
- 3 Non-contiguous memory allocation
- 4 Virtual memory

Contiguous?

Here an executable is loaded in **one inseparable block of data** in memory

Here ...

- The *loader* looks for the **best free area in memory** to load
- Once the *process* is terminated the **block of data is freed**



Loading a new program in memory consists in ...

- ... finding a **big enough free area** to load the the program
- ... **minimizing the wasted space** in main memory
- ... **minimizing the time spent** to find the best place

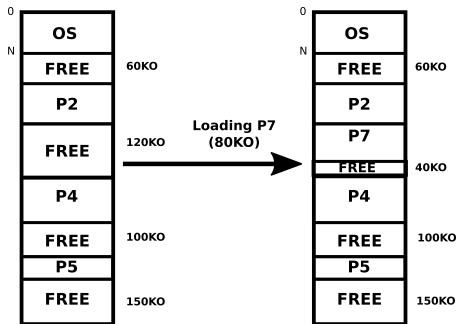
Loading a new program in memory consists in ...

- ... finding a **big enough free area** to load the the program
- ... **minimizing the wasted space** in main memory
- ... **minimizing the time spent** to find the best place

Three strategies to find a contiguous free area:

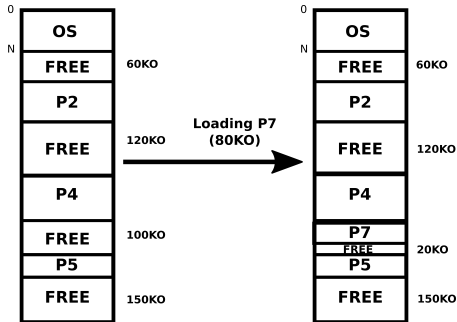
- 1 The "*First free area*" strategy
- 2 The "*Best free area*" strategy
- 3 The "*Worst free area*" strategy

The "*First free area*" strategy:



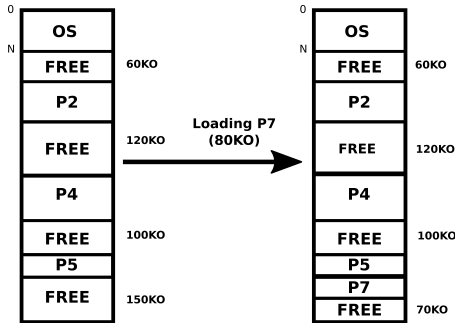
- Advantage: **Fastest approach**
- BUT: **Not designed to optimize the main memory usage (40KO)**

The "*Best free area*" strategy:



- Advantage: **best main memory usage** (20KO)
- BUT: **slow** (need to examine the memory to have the best fitting area)

The "*Worst free area*" strategy:

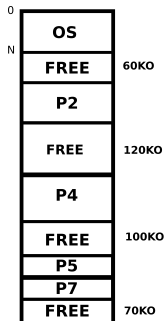


- Advantage: create **bigger memory fragments** (can be used after!)
- BUT: **slow** (need to examine the memory to have the best fitting area)

Fragmentation?

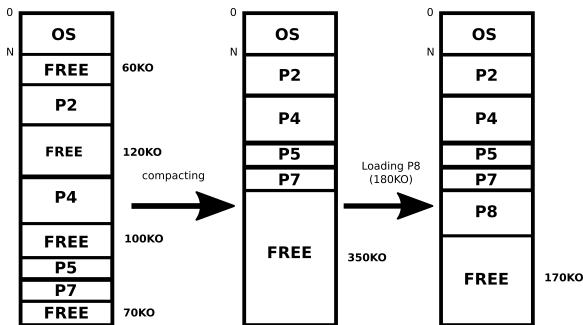
Memory gets *fragmented* rapidly:

- Process **creations and terminations** fragment the memory
- When too fragmented the memory **may become unusable!**



**IMPOSSIBLE to load
P8 (180KO)! Even if
 $60+120+100+70=350\text{KO}$ free**

One solution is to **compact** the *physical memory* first:



BUT: it is an **extremely slow** and **not optimized** approach

- 1 Introduction
- 2 Contiguous allocation of the physical memory
- 3 Non-contiguous memory allocation**
- 4 Virtual memory

Non-contiguous?

Here we consider that one *program* = **several parts**

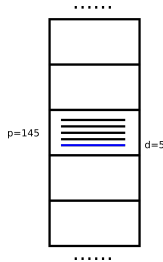
Here ...:

- each part has an **equal size** and is called: a **page**
- the parts **can be at very different locations** in main memory
- the *physical memory* is divided in **empty cases of equal sizes** (512 to 8192 bytes)

We call this memory allocation mechanism: the **pagination**

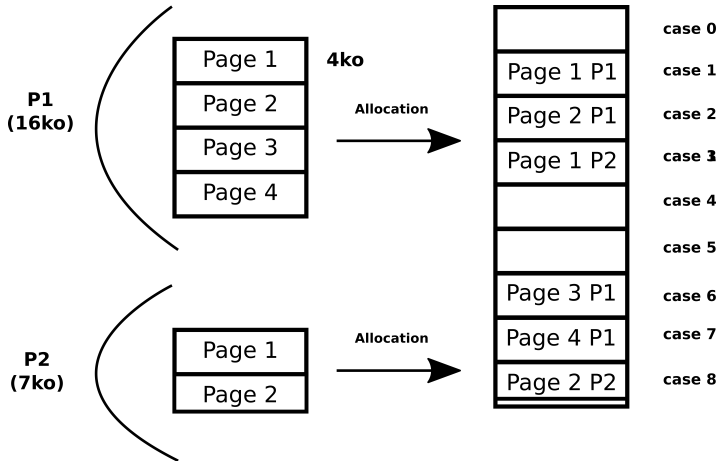
Because the *physical memory* is divided in pages the memory is ...

- ... accessed giving a page index p and a displacement d inside it
 - ▶ example: $\langle p, d \rangle$



Note

paginated addresses are converted to *physical addresses* later (for CPU)



Note

With *pagination* the *memory fragmentation* is not a problem!

To know where are all program pages we have: the *table of pages*.
Example:

case	page	program
0	-1	
1	1	P1
2	2	P1
3	1	P2
4	-1	
5	-1	
6	3	P1
7	4	P1
8	2	P2

Note
-1 in the *table of pages* just mean that this case is not occupied by a page

- 1 Introduction
- 2 Contiguous allocation of the physical memory
- 3 Non-contiguous memory allocation
- 4 Virtual memory

It comes from an observation: **not all pages are always necessary**:

- Only a small amount of a pages need to be in physical memory
- The rest can stay in mass storage (in the virtual memory)

Here our *table of pages* has another column (V)²:

case	page	program	V
0	-1		0
1	1	P1	1
-1	2	P1	0
-1	1	P2	0
4	-1		0
5	-1		0
6	3	P1	1
-1	4	P1	0
8	2	P2	1

²When V=1: page in *physical memory*, V=0: page in *virtual memory*

case	page	program	V
0	-1		0
1	1	P1	1
-1	2	P1	0
-1	1	P2	0
4	-1		0
5	-1		0
6	3	P1	1
-1	4	P1	0
8	2	P2	1

Note

When we have $V=0$ for a page: the system **detects it in advance** and loads the page in main memory so that the *CPU* can execute smoothly the instructions in the page, without stopping.

There are two approaches to attribute the number of cases per program:

- ① Giving the same number of cases per program
 - ▶ BUT: a small process can have as many as a big process
- ② Giving a proportionate to the size number of cases
 - ▶ Advantage: bigger processes have more cases

Indeed, processes in **main memory** may have very different sizes...

Note

A process has an initial **number of cases** used to load its active pages:

- number of cases per program $<$ total number of program pages

When a *process* has used all its available cases:

- a **new case must be found** for loading the next page!
- a case can be found by **replacing an existing page!**
- either in its **own initially attributed cases**
- or in **others' attributed cases**

When a *process* has used all its available cases:

- a **new case must be found** for loading the next page!
- a case can be found by **replacing an existing page!**
- either in its **own initially attributed cases**
- or in **others' attributed cases**

There are several algorithms to find the ideal *victim page*:

- The *FIFO* replacement algorithm
- The *Least Recently Used* replacement algorithm
- The *second chance* algorithm
- The *Least Frequently Used* replacement algorithm
- The *Most Frequently Used* replacement algorithm

- The *FIFO* replacement algorithm:

The oldest loaded page is replaced

- The *Least Recently Used* replacement algorithm:

The least used page is replaced. This algorithm is one of the most used. But, it is hard to implement and require extra hardware.

- The *second chance* algorithm:

This algorithm is an approximation of the LRU algorithm. Less complex to implement.

- The *Least Frequently Used* replacement algorithm:

The least frequently used page is replaced. But it raises problems when a large number of pages are loaded in a short time: they risk to be replaced first!

- The *Most Frequently Used* replacement algorithm

Here, the most frequently used page is replaced. There is an opposite logic here: we consider that a page that is not used a lot needs to stay in memory.